Experiment 3: Probability Distributions and Bayes' Theorem

1. AIM

To generate and visualize Bernoulli, Binomial, Poisson, Uniform, and Gaussian probability distributions using NumPy and SciPy. To apply Bayes' theorem to calculate the probability that a patient actually has a disease given a positive test result, verify the probability using a simulation of 100,000 patients, and analyze how the posterior probability changes with disease prevalence.

2. MATHEMATICAL FORMULAS

1. Bernoulli Distribution

$$\boxed{P(X = x) = p^x(1-p)^{1-x}}$$

where $x \in \{0, 1\}$.

2. Binomial Distribution

$$\boxed{P(X = k) = \binom{n}{k} p^k (1-p)^{n-k}}$$

where $k = 0, 1, \ldots, n$.

3. Poisson Distribution

$$\boxed{P(X = k) = \frac{e^{-\lambda}\lambda^k}{k!}}$$

where $k = 0, 1, 2, \ldots$.

4. Uniform Distribution

$$\boxed{f(x) = \frac{1}{b-a}, \quad a \le x \le b}$$

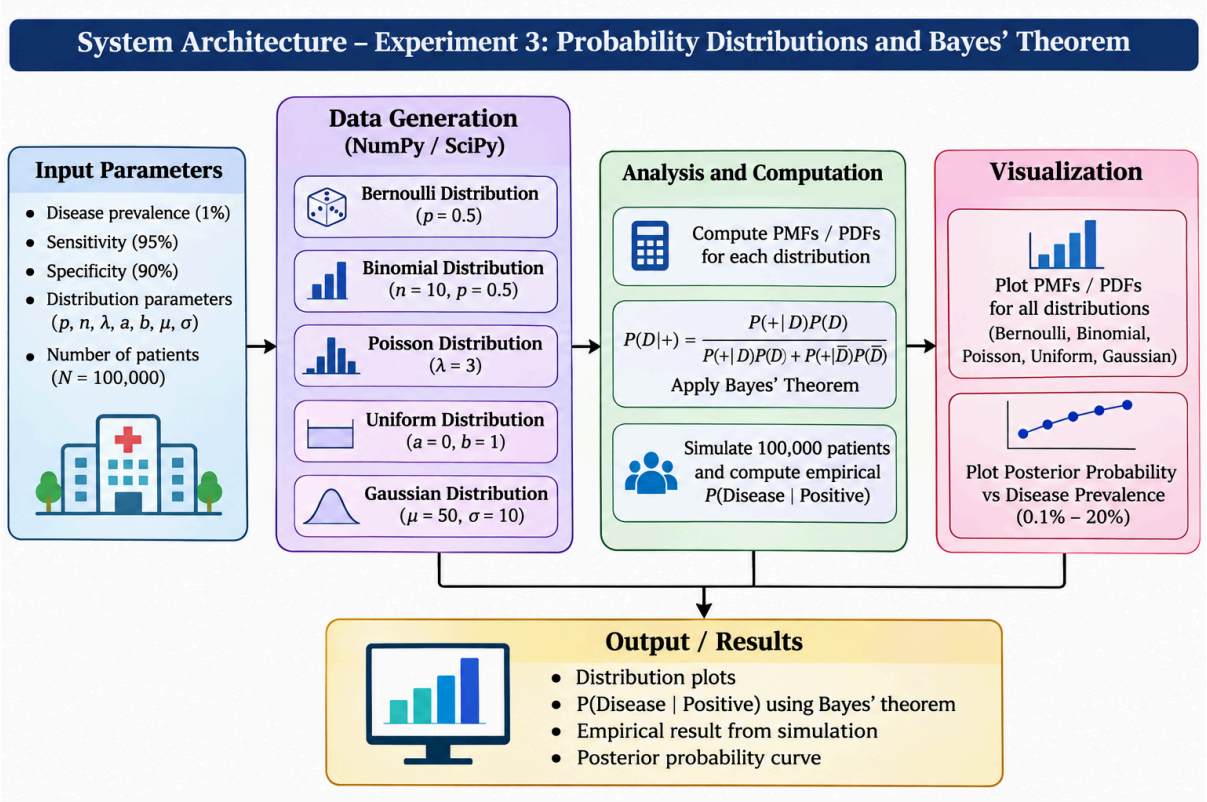
5. Gaussian (Normal) Distribution

$$\boxed{f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}}$$

where μ is the mean and σ is the standard deviation.

6. Bayes' Theorem

$$\boxed{P(D \mid +) = \frac{P(+ \mid D)P(D)}{P(+ \mid D)P(D) + P(+ \mid \bar{D})P(\bar{D})}}$$



CODE

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats

np.random.seed(42)

# 1. Generate probability distributions
N = 100000

# Bernoulli Distribution
p = 0.5
bernoulli = np.random.binomial(1, p, N)

# Binomial Distribution
n, p_bin = 10, 0.5
binomial = np.random.binomial(n, p_bin, N)

# Poisson Distribution
lam = 3
poisson = np.random.poisson(lam, N)
```

```
# Uniform Distribution
uniform = np.random.uniform(0, 1, N)

# Gaussian Distribution
mu, sigma = 50, 10
gaussian = np.random.normal(mu, sigma, N)

# 2. Plot distributions and theoretical PMFs/PDFs
fig, axes = plt.subplots(2, 3, figsize=(15, 9))

# Bernoulli
x = np.array([0, 1])
axes[0, 0].bar(
    x, stats.bernoulli.pmf(x, p),
    color='skyblue', edgecolor='black'
)
axes[0, 0].set_title("Bernoulli Distribution")
axes[0, 0].set_xlabel("x")
axes[0, 0].set_ylabel("Probability")

# Binomial
x = np.arange(0, n + 1)
axes[0, 1].bar(
    x, stats.binom.pmf(x, n, p_bin),
    color='salmon', edgecolor='black'
)
axes[0, 1].set_title("Binomial Distribution")
axes[0, 1].set_xlabel("x")
axes[0, 1].set_ylabel("Probability")

# Poisson
x = np.arange(0, 15)
axes[0, 2].bar(
    x, stats.poisson.pmf(x, lam),
    color='lightgreen', edgecolor='black'
)
axes[0, 2].set_title("Poisson Distribution")
axes[0, 2].set_xlabel("x")
axes[0, 2].set_ylabel("Probability")

# Uniform
x = np.linspace(0, 1, 100)
axes[1, 0].plot(
```

```
    x, stats.uniform.pdf(x),
    color='purple'
)
axes[1, 0].set_title("Uniform Distribution")
axes[1, 0].set_xlabel("x")
axes[1, 0].set_ylabel("Density")

# Gaussian
x = np.linspace(20, 80, 200)
axes[1, 1].plot(
    x, stats.norm.pdf(x, mu, sigma),
    color='orange'
)
axes[1, 1].set_title("Gaussian Distribution")
axes[1, 1].set_xlabel("x")
axes[1, 1].set_ylabel("Density")

axes[1, 2].axis('off')

plt.tight_layout()
plt.show()

# 3. Bayes' Theorem
prevalence = 0.01
sensitivity = 0.95
specificity = 0.90
false_positive_rate = 1 - specificity

numerator = sensitivity * prevalence

denominator = (
    sensitivity * prevalence
    + false_positive_rate * (1 - prevalence)
)

posterior = numerator / denominator

print("Bayes' Theorem Result")
print("Disease Prevalence:", prevalence)
print("Sensitivity:", sensitivity)
print("Specificity:", specificity)
print("P(Disease | Positive):", round(posterior, 5))
print("Posterior Probability (%):", round(posterior * 100, 2))
```

```
# 4. Simulate 100,000 patients
disease = np.random.random(N) < prevalence

test_positive = np.where(
    disease,
    np.random.random(N) < sensitivity,
    np.random.random(N) < false_positive_rate
)

positive_count = np.sum(test_positive)

if positive_count > 0:
    empirical_posterior = np.sum(
        disease & test_positive
    ) / positive_count
else:
    empirical_posterior = 0

print("\nSimulation Result")
print("Total Patients:", N)
print("Positive Test Results:", positive_count)
print("Actual Diseased Patients:", np.sum(disease))
print("Empirical P(Disease | Positive):",
      round(empirical_posterior, 5))
print("Empirical Probability (%):",
      round(empirical_posterior * 100, 2))

# 5. Posterior probability for different prevalence values
prevalence_values = np.linspace(0.001, 0.20, 200)

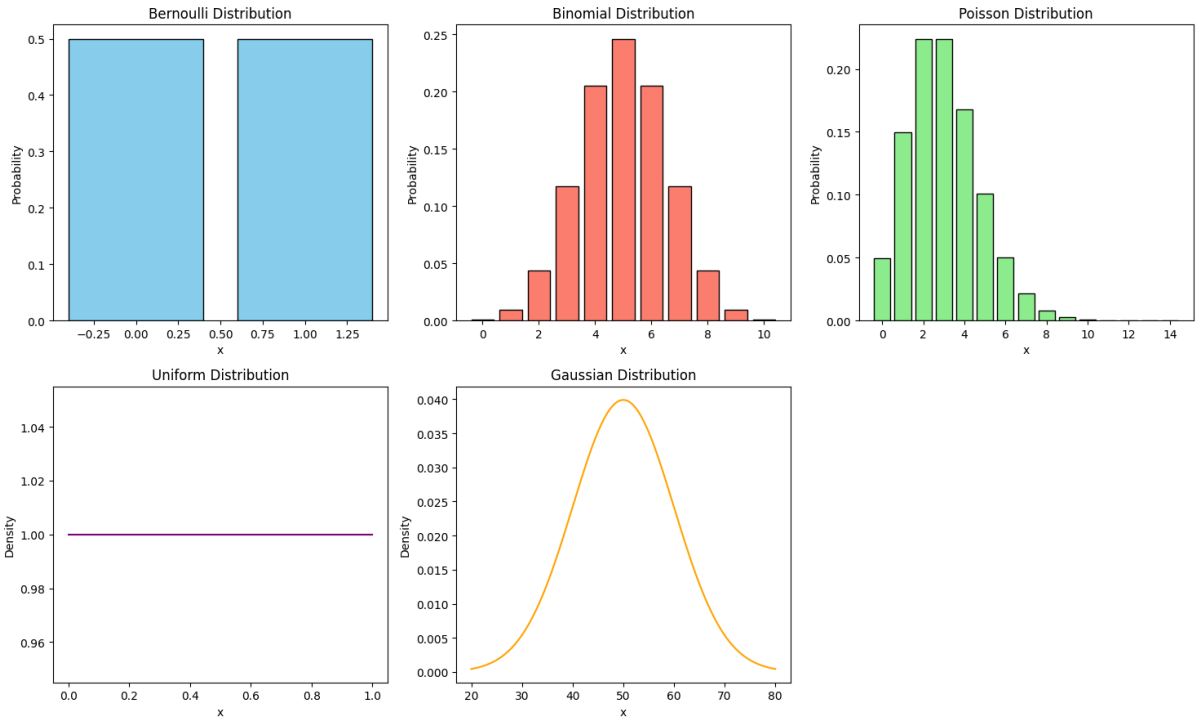
posteriors = (
    sensitivity * prevalence_values
) / (
    sensitivity * prevalence_values
    + false_positive_rate * (1 - prevalence_values)
)

plt.figure(figsize=(9, 5))
plt.plot(
    prevalence_values * 100,
    posteriors * 100,
    color='blue'
```

```
)

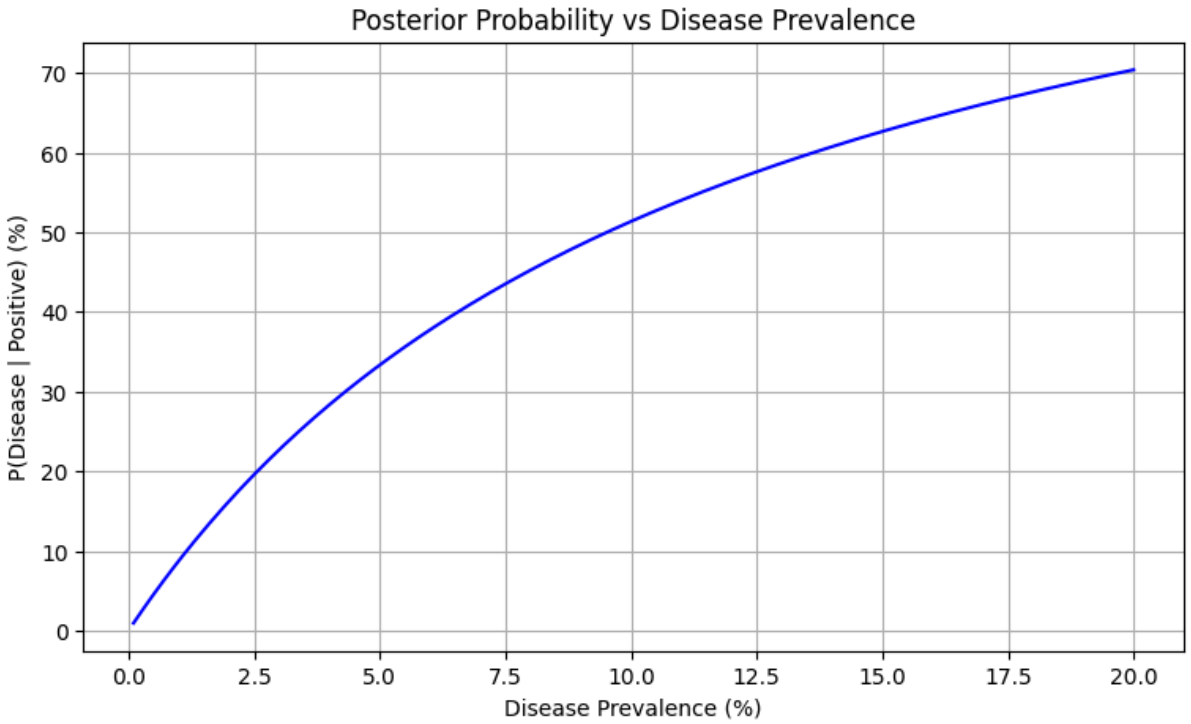
plt.xlabel("Disease Prevalence (%)")
plt.ylabel("P(Disease | Positive) (%)")
plt.title("Posterior Probability vs Disease Prevalence")
plt.grid(True)
plt.show()
```

OUTPUT




```
Bayes' Theorem Result
Disease Prevalence: 0.01
Sensitivity: 0.95
Specificity: 0.9
P(Disease | Positive): 0.08756
Posterior Probability (%): 8.76

Simulation Result
Total Patients: 100000
Positive Test Results: 10950
Actual Diseased Patients: 1051
Empirical P(Disease | Positive): 0.09205
Empirical Probability (%): 9.21
```


Posterior Probability vs Disease Prevalence
P(Disease | Positive) (%)
0
10
20
30
40
50
60
70
0.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5
20.0
Disease Prevalence (%)